

Falta de constancia en hábitos de actividad física

CC3090 – Ingeniería de Software 1

Camila Sandoval **#24358**

Emily Góngora **#24308**

Alejandra Sierra **#24405**

Martin Villatoro **#24033**

Alejandro Pérez **#24136**

Esteban de la Peña **#24171**

I. Resumen

El proyecto que estamos realizando consiste en el desarrollo de una app enfocada en la promoción de la consistencia en la actividad física mediante challenges personalizables. La solución se plantea en el contexto de un problema actual: la dificultad que gente enfrenta para mantener hábitos de ejercicio debido a faltas de motivación, acompañamiento adecuado y flexibilidad. La propuesta busca responder a esta necesidad mediante una app que permita a los usuarios crear y unirse a diferentes challenges, registrar su progreso y participar en espacios sociales y comunidades que ayudan a fomentar el compromiso.

En este corte, se realizó principalmente la implementación técnica del sistema, incluyendo la definición de la arquitectura modular por capas, la integración de tecnologías y el diseño de un sistema escalable.

Como objetivo, se busca construir una solución funcional y eficiente que puede soportar varios usuarios, gestionar grandes volúmenes de datos y garantizar tiempos de respuesta adecuados, asegurando una experiencia fluida.

II. Introducción

El proyecto está dirigido a personas que quieren empezar, mejorar o mantener sus hábitos de ejercicio. La solución que proponemos es una aplicación donde los usuarios pueden registrar sus entrenamientos, ver su progreso, y crear o unirse a retos. Además, pueden interactuar con otros usuarios dentro de la app, ya sea compartiendo sus avances, uniéndose a retos con sus amigos o chateando con ellos.

La idea principal de esta app es los challenges que el usuario puede unirse y la adaptabilidad que estos tienen para cada persona. El proyecto busca resolver la falta de consistencia que personas tienen con el ejercicio. Para esto, la app permitirá que crear o se unirse a challenges, registrar su progreso con datos y fotos, interactuar con otros usuarios y recibir notificaciones. Todo esto con el objetivo de ayudar a mantener la motivación y crear hábitos saludables de una forma más fácil y constante.

III. Desarrollo

1. Prototipos

a. Versión 1

Prototipo:

https://www.canva.com/design/DAGzw3tJmIU/zagsyync26Pq6SHrUCFbRQ/view?utm_content=DAGzw3tJmIU&utm_campaign=designshare&utm_medium=link2&utm_source=uniquelinks&utm_id=h651b7e713b

Evidencia:

Usuario 1: Jovenes en etapa academica

Usuario 2: Adultos con responsabilidades multiples

Usuario 3: Adultos mayores

Nombre	Tipo de usuario	Lo que le gustó	Lo que no le gustó
<i>Perfil 1: Personas que no hacen ejercicio pero quieren empezar</i>			
Santiago Chacón	Usuario 1	Que explorar diferentes retos era bastante fácil.	No entendía bien donde podía visualizar la rutina después de unirse a un challenge.
Ian Quan	Usuario 1	Le gustó la incorporación de los “spaces” en la pantalla de chats.	No entendía bien donde crear challenges porque el botón era muy pequeño.
Pablo Oliva	Usuario 2	Le gustó el botón para subir fotos a los challenges.	La pantalla de invitar amigos era poco llamativa.
Henry Funes	Usuario 3	Le gustó el diseño y colores de la pantalla de challenges.	En la pantalla de Home no le gustó que no se enseñaba cuanto tiempo le faltaba al challenge.
<i>Perfil 2: Personas que hacen ejercicio de forma intermitente o inconsistente</i>			
Natalia Gaitán	Usuario 1	Le gustó que solo se pueden subir fotos tomadas desde la app.	No le gustó que no se podía interactuar con los posts de amigos.
Sidney Palala	Usuario 1	Le gustaron los badges de actividades físicas en el perfil de usuario.	No le gustó que el botón de chats siempre se mostraba aún al deslizar. Le estorbó.
Fernanda de Sandoval	Usuario 2	Les gustaron los colores usados en el diseño de la app.	No le gustó que en la pantalla de exploración de challenges, no se presentaba mucha

			información sobre ellos.
Margot Cano	Usuario 3	Le gustó que la descripción de los challenges es clara.	No le gustó que no podía ver en donde eran adaptables las rutinas (casa, gym, al aire libre).
<i>Perfil 3: Personas constantes y competitivas en el ejercicio</i>			
Javier Castillo	Usuario 1	Le gustó que se podían incluir rutinas en los challenges.	No le gustó que la pantalla de perfil no mostraba logros o trofeos.
Gianpaolo Mencos	Usuario 1	Le gustó la visualización de rachas de amigos en la pantalla de inicio.	No le gustó que el botón de configuración en la pantalla de inicio no tenía un propósito claro.
Marlon Navarro	Usuario 2	Le gustaron las dos vistas para ver el progreso de los challenges.	No le gustó que la pantalla de descripción de challenge no indica quien hizo el challenge.
Karina Cabrera	Usuario 3	Le gustó	No le gustó que en ninguna pantalla se le preguntaron sus objetivos.

b. Versión 2:Prototipo:

https://www.canva.com/design/DAHDtWVeDI0/q439DvWGbZnEjyPmPSwovA/view?utm_content=DAHDtWVeDI0&utm_campaign=designshare&utm_medium=link2&utm_source=uniquelinks&utlId=h06dc271fc5

Evidencia:**Usuario 1:** Jóvenes en etapa académica**Usuario 2:** Adultos con responsabilidades múltiples**Usuario 3:** Adultos mayores

Nombre	Tipo de usuario	Lo que le gustó	Lo que no le gustó
<i>Perfil 1: Personas que no hacen ejercicio pero quieren empezar</i>			
Santiago Chacón	Usuario 1	Le gustó que hay un área para personalizar su estilo de hacer fitness.	No le gustó que algunos botones se ven un poco cercas entre ellos
Ian Quan	Usuario 1	Le gustó poder ver fácilmente en que challenges está participando actualmente.	No le gustó que algunas partes de la app requieren explorar un poco
Pablo Oliva	Usuario 2	Le gustó que es fácil descubrir nuevos challenges	No le gustó que algunas funciones no son evidentes en el inicio
Henry Funes	Usuario 3	Le gustó que el progreso dentro de los challenges es fácil de identificar	No le gustó que algunos textos se ven pequeños
<i>Perfil 2: Personas que hacen ejercicio de forma intermitente o inconsistente</i>			
Natalia Gaitán	Usuario 1	Le gustó que se siente bien organizado y no “sobre estimulado”	No le gustó que algunas cosas se ven un poco escondidas
Sidney Palala	Usuario 1	Le gustó que la app se siente bastante clara para seguir los retos del challenge	No le gustó que se ve oscuro
Fernanda de Sandoval	Usuario 2	Le gustó que te da una rutina para seguir	No le gustó algunas combinaciones de colores

Margot Cano	Usuario 3	Le gustó que se puede visualizar en donde se pueden hacer los challenges (si se puede ser desde casa o en el gym)	No le gustó que algunas pantallas se ven tan llenas
<i>Perfil 3: Personas constantes y competitivas en el ejercicio</i>			
Javier Castillo	Usuario 1	Le gustó que hay una área para ver tus rutinas	No le gustó que a veces no sabe donde irse
Gianpaolo Mencos	Usuario 1	Le gustó que se siente motivadora estar/empezar en un challenge	No le gustó que algunas cosas se ven fuera de lugar
Marlon Navarro	Usuario 2	Le gustó que se agregó un “rest day ” button así no tienen que dejar sus challenges	No le gustó que le gustaría que fuera más clara en general.
Karina Cabrera	Usuario 3	Le gustó que hay una pantalla de métricas donde las pueden mantener un registro de sus reps y lo demás.	No le gustó que hay unas cosas que no entiende tan facil

Cambios:

- Agregamos una pantalla de métricas
- Agregamos un rest day button así el usuario no tiene que dejar sus challenges cuando no pudieron cumplir con el por alguna razón. Además la posibilidad de programar rest days que no influyen en sus challenges
- Agregamos Location tags donde dice si se puede realizar los challenges en tu casa, el gym, al aire libre, etc..
- Agregamos las pantallas de “Create Challenge”
- Agregamos como se ve las rutinas de los challenges
- Agregamos una pantalla de donde se visualizará tus challenges y rutinas
- Cambiamos el botón de “Create Challenges”
- Quitamos cuanto tiempo faltaba en los challenges del Home page
- Agregamos la posibilidad de interactuar con posts
- Agregamos un “Search by Category/Trending Search”
- Agregamos logros/trofeos
- Cambiamos en los chats Friends a Messages para evitar confusiones¹
- Agregamos pantallas de chats; como se vería crear grupos o “spaces”, chats,

2. Requisitos funcionales

Acceso al sistema

RF1. Registro de usuario

El sistema debe permitir a un usuario crear una cuenta proporcionando correo electrónico y contraseña válidos.

Historia de usuario asociada: HU1 – Registrarse en la aplicación.

RF2. Inicio de sesión

El sistema debe permitir a los usuarios autenticarse en la aplicación mediante sus credenciales registradas.

Historia de usuario asociada: HU2 – Iniciar sesión.

Configuración inicial del perfil

RF3. Selección de tipo de perfil

El sistema debe permitir al usuario seleccionar un tipo de perfil (principiante, intermitente o constante) para personalizar su experiencia.

Historia de usuario asociada: HU3 – Seleccionar tipo de perfil.

RF4. Definición de objetivos personales

El sistema debe permitir al usuario definir objetivos personales relacionados con su actividad física.

Historia de usuario asociada: HU4 – Definir objetivos personales.

RF5. Edición de preferencias de actividad física

El sistema debe permitir al usuario editar sus preferencias de actividad física para adaptar la aplicación a sus necesidades.

Historia de usuario asociada: HU5 – Editar preferencias de actividad física.

Exploración de retos

RF6. Exploración de retos por categoría

El sistema debe permitir a los usuarios explorar retos disponibles filtrados por categorías como cardio, fuerza o flexibilidad.

Historia de usuario asociada: HU6 – Explorar retos por categoría.

Creación y participación en retos

RF7. Creación de retos personalizados

El sistema debe permitir a los usuarios crear retos personalizados de actividad física.

Historia de usuario asociada: HU7 – Crear reto personalizado.

RF8. Definición de duración y frecuencia del reto

El sistema debe permitir establecer la duración y frecuencia de un reto durante su creación.

Historia de usuario asociada: HU8 – Definir duración y frecuencia del reto.

RF9. Selección del lugar de realización del reto

El sistema debe permitir definir el lugar donde se realizará el reto (hogar, gimnasio, exterior, etc.).

Historia de usuario asociada: HU9 – Elegir lugar de realización del reto.

RF10. Unirse a retos existentes

El sistema debe permitir a los usuarios unirse a retos previamente creados por otros usuarios.

Historia de usuario asociada: HU10 – Unirse a un reto existente.

RF11. Publicación de retos por creadores de contenido

El sistema debe permitir a los creadores de contenido publicar retos disponibles para otros usuarios.

Historia de usuario asociada: HU23 – Crear y publicar un reto

RF12. Agregar contenido explicativo a los retos

El sistema debe permitir agregar descripciones, instrucciones o videos a los retos creados.

Historia de usuario asociada: HU24 – Agregar descripciones o videos al reto.

RF13. Visualización del impacto de los retos creados

El sistema debe permitir a los creadores visualizar la cantidad de usuarios que se han unido a sus retos.

Historia de usuario asociada: HU25 – Visualizar impacto del reto creado.

Registro y seguimiento del progreso

RF14. Registro de repeticiones por ejercicio

El sistema debe permitir registrar la cantidad de repeticiones realizadas por ejercicio.

Historia de usuario asociada: HU11 – Registrar repeticiones

RF15. Registro del peso utilizado en ejercicios

El sistema debe permitir registrar el peso utilizado en los ejercicios realizados.

Historia de usuario asociada: HU12 – Registrar peso utilizado

RF16. Subida de fotografías de progreso

El sistema debe permitir a los usuarios subir fotografías para documentar su progreso.

Historia de usuario asociada: HU13 – Subir fotografías de progreso

RF17. Visualización del historial de actividades

El sistema debe permitir a los usuarios consultar el historial de sus actividades físicas registradas.

Historia de usuario asociada: HU14 – Ver historial de actividades.

RF18. Reconocimiento del esfuerzo del usuario

El sistema debe reconocer el esfuerzo del usuario aunque no complete completamente un reto.

Historia de usuario asociada: HU15 – Reconocimiento del esfuerzo.

Comunidad e interacción social

RF19. Participación en espacios comunitarios asociados a retos

El sistema debe permitir que los usuarios participen en espacios comunitarios relacionados con retos o intereses compartidos, donde puedan comunicarse e

interactuar con otros participantes.

Historia de usuario asociada: HU16 – Participar en grupos asociados a un reto.

RF20. Compartir progreso mediante publicaciones

El sistema debe permitir a los usuarios compartir su progreso mediante publicaciones que incluyan imágenes y descripciones dentro de la comunidad.

Historia de usuario asociada: HU17 – Compartir progreso en galería grupal.

RF21. Configuración de privacidad de publicaciones

El sistema debe permitir a los usuarios definir la visibilidad de sus publicaciones, pudiendo establecerlas como visibles solo para seguidores o privadas.

Historia de usuario asociada: HU18 – Configurar privacidad de publicaciones.

RF22. Interacción con publicaciones y usuarios

El sistema debe permitir a los usuarios interactuar con el contenido de otros miembros mediante acciones como reaccionar a publicaciones o enviar mensajes dentro de los espacios comunitarios o conversaciones privadas.

Historia de usuario asociada: HU19 – Interactuar con miembros del grupo.

Adaptabilidad del sistema

RF23. Adaptación de duración de sesiones

El sistema debe permitir a los usuarios modificar la duración de sus sesiones cuando cambien sus rutinas.

Historia de usuario asociada: HU20 – Adaptar duración de sesiones.

RF24. Sugerencias personalizadas de ejercicios

El sistema debe ofrecer sugerencias de ejercicios basadas en el nivel y objetivos del usuario.

Historia de usuario asociada: HU21 – Recibir sugerencias personalizadas

RF25. Modificación de retos en curso

El sistema debe permitir modificar un reto que ya se encuentre en curso.

Historia de usuario asociada: HU22 – Modificar reto en curso.

Moderación del sistema

RF26. Revisión de contenido comunitario

El sistema debe permitir a los moderadores revisar las publicaciones realizadas por los usuarios.

Historia de usuario asociada: HU26 – Revisar publicaciones

RF27. Eliminación de contenido inapropiado

El sistema debe permitir a los moderadores eliminar contenido que infrinja las normas de la comunidad.

Historia de usuario asociada: HU27 – Eliminar contenido inapropiado.

Sistema de notificaciones

RF28. Envío de recordatorios de retos

El sistema debe enviar recordatorios a los usuarios sobre los retos activos en los que participan

Historia de usuario asociada: HU28 – Recibir recordatorios de retos activos

RF29. Notificaciones de actividad

El sistema debe enviar notificaciones cuando exista actividad relevante en publicaciones, retos o interacciones sociales.

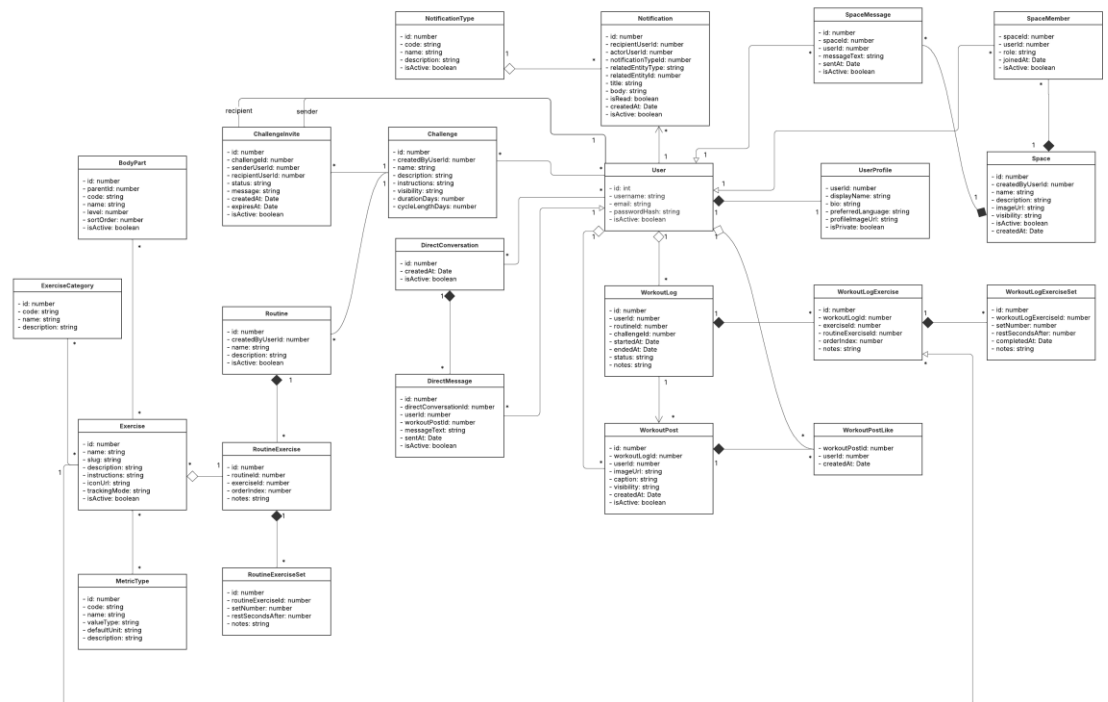
Historia de usuario asociada: HU29 – Recibir notificaciones de actividad.

Modificación de requisitos no funcionales:

Requisitos No Funcionales	Categoría	Forma en que se medirá su cumplimiento
El servidor deberá ejecutarse en entornos Linux y ser desplegable en servicios de nube pública (ej. AWS o similares). <u>Modificación:</u> El backend deberá ser portable y ejecutable en entornos Linux mediante contenedores Docker, permitiendo su eventual despliegue en servicios de nube pública	Requerimientos de Software	Se validará mediante despliegue exitoso en un entorno de pruebas. <u>Modificación:</u> Se validará ejecutando el sistema mediante Docker en un entorno Linux y verificando el acceso correcto a la API.

3. Clases preliminares:

a. Diagrama de clases UML



https://lucid.app/lucidchart/96bf0cb8-f842-4255-918e-c1fb70dd9a46/edit?viewport_loc=-598%2C-449%2C4897%2C1965%2CHWEp-vi-RSFO&invitationId=inv_bc854d77-6101-43a3-95ca-207d2c87a598

b. Descripción de las clases

Las clases fueron identificadas a partir de los requerimientos no funcionales y se organizan siguiendo una arquitectura modular por capas (explicada en la sección de Selección de tecnologías). Cada módulo del sistema agrupa las clases relacionadas con una funcionalidad específica, mientras que dentro de cada módulo las clases se distribuyen según su responsabilidad en distintas capas del backend. Se distinguen clases pertenecientes a la capa de presentación (controllers), capa de lógica de negocio (services), capa de acceso a datos (repositories) y capa de persistencia (entities).

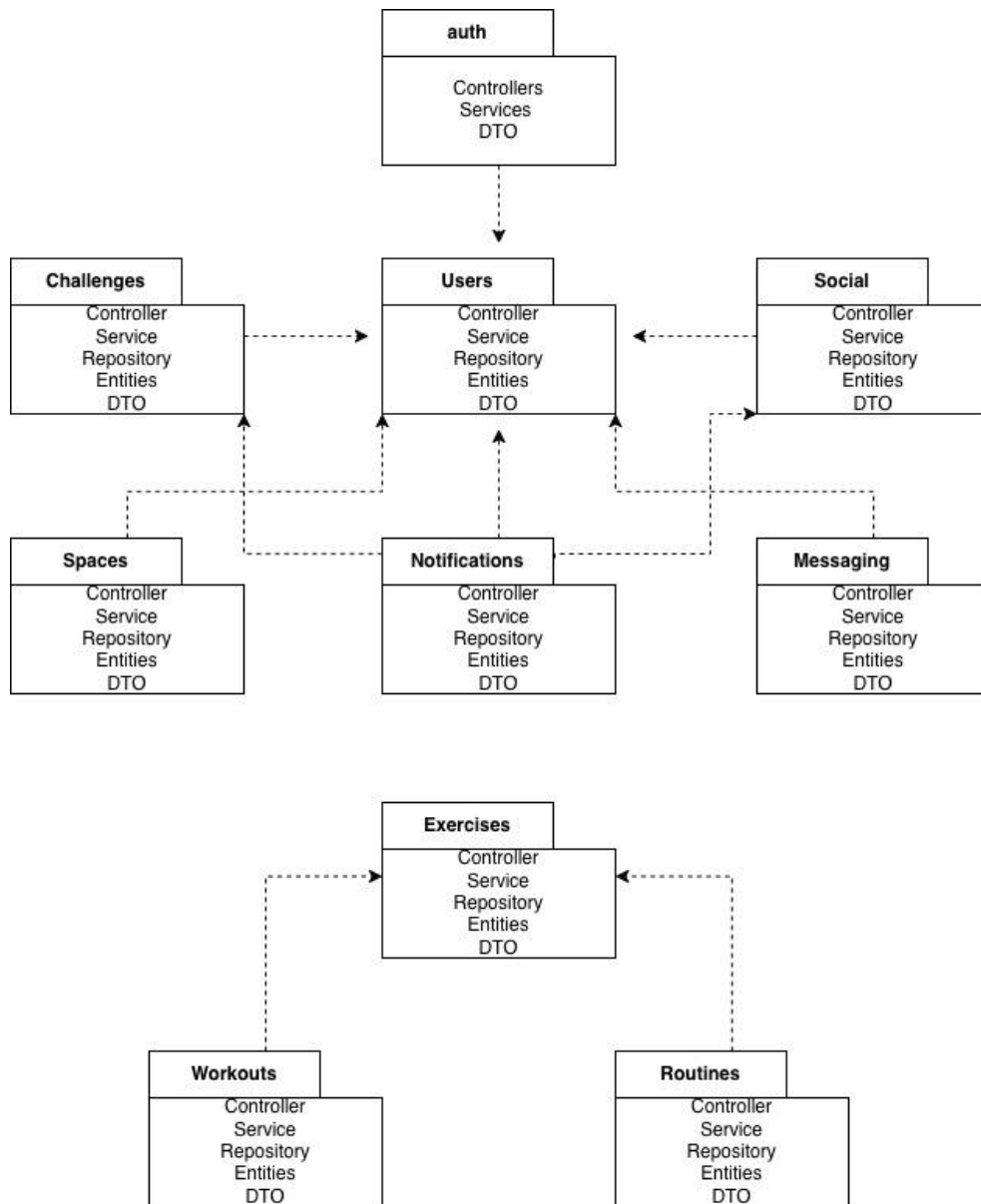
Clase	Capa	Requisito Funcional	Descripción
AuthController	Presentación (Controller)	RF1, RF2	Gestiona las solicitudes HTTP relacionadas con autenticación, como registro e inicio de sesión.
AuthService	Lógica de negocio (Service)	RF1, RF2	Implementa la lógica de autenticación, validación de credenciales y generación de tokens JWT.
UserController	Presentación (Controller)	RF3, RF4, RF5, RF17	Permite consultar y actualizar información del perfil del usuario.
UserService	Lógica de negocio (Service)	RF3, RF4, RF5, RF17, RF18, RF23, RF24	Gestiona la lógica del perfil del usuario, objetivos personales, preferencias y recomendaciones.
UserRepository	Acceso a datos (Repository)	RF1, RF2, RF3, RF4, RF5	Maneja las consultas y operaciones de base de datos relacionadas con los usuarios.
UserEntity	Persistencia (Entity)	RF1, RF2	Representa la tabla de usuarios que almacena credenciales y estado de la cuenta.

UserProfileEntity	Persistencia (Entity)	RF3, RF4, RF5	Representa la información pública y configuración del perfil del usuario.
ExerciseController	Presentación (Controller)	RF6, RF24	Permite explorar ejercicios disponibles y obtener sugerencias de actividad física.
ExerciseService	Lógica de negocio (Service)	RF6, RF14, RF15, RF24	Gestiona la lógica de exploración y selección de ejercicios y métricas asociadas.
ExerciseRepository	Acceso a datos (Repository)	RF6	Permite consultar el catálogo de ejercicios almacenado en la base de datos.
ExerciseEntity	Persistencia (Entity)	RF6, RF14, RF15	Representa los ejercicios disponibles en el sistema y su información descriptiva.
ChallengeController	Presentación (Controller)	RF6, RF7, RF8, RF9, RF10, RF11, RF12, RF13, RF25	Gestiona solicitudes relacionadas con retos, como exploración, creación y participación.
ChallengeService	Lógica de negocio (Service)	RF6, RF7, RF8, RF9, RF10, RF11, RF12, RF13, RF25	Implementa la lógica para crear retos, definir duración, frecuencia y gestionar participantes.
ChallengeRepository	Acceso a datos (Repository)	RF7, RF10	Maneja el almacenamiento y consulta de retos en la base de datos.
ChallengeEntity	Persistencia (Entity)	RF7, RF8, RF11, RF12	Representa los retos creados por los usuarios dentro del sistema.
RoutineController	Presentación (Controller)	RF23, RF24	Permite gestionar rutinas de entrenamiento y su configuración.
RoutineService	Lógica de negocio (Service)	RF23, RF24	Gestiona la lógica para modificar rutinas y adaptar la duración de sesiones.
RoutineRepository	Acceso a datos (Repository)	RF23	Permite consultar y guardar rutinas en la base de datos.
RoutineEntity	Persistencia (Entity)	RF23	Representa las rutinas de entrenamiento reutilizables del sistema.

WorkoutController	Presentación (Controller)	RF14, RF15, RF17	Gestiona el registro de ejercicios realizados y la consulta del historial de actividades.
WorkoutService	Lógica de negocio (Service)	RF14, RF15, RF17, RF18	Implementa la lógica para registrar métricas de ejercicios y analizar progreso.
WorkoutRepository	Acceso a datos (Repository)	RF14, RF15	Maneja el almacenamiento de sesiones de entrenamiento realizadas.
WorkoutLogEntity	Persistencia (Entity)	RF14, RF15, RF17	Representa una sesión de entrenamiento registrada por el usuario.
SocialController	Presentación (Controller)	RF16, RF20, RF21, RF22	Gestiona las publicaciones de progreso y las interacciones sociales entre usuarios.
SocialService	Lógica de negocio (Service)	RF16, RF20, RF21, RF22	Implementa la lógica para compartir progreso, configurar privacidad e interactuar con contenido.
SocialRepository	Acceso a datos (Repository)	RF20	Permite almacenar publicaciones y reacciones en la base de datos.
WorkoutPostEntity	Persistencia (Entity)	RF16, RF20, RF21	Representa publicaciones de progreso con imágenes y descripciones.
SpaceController	Presentación (Controller)	RF19, RF22	Gestiona la interacción dentro de espacios comunitarios asociados a retos o intereses.
SpaceService	Lógica de negocio (Service)	RF19, RF22	Implementa la lógica de comunicación y gestión de miembros dentro de espacios comunitarios.
SpaceRepository	Acceso a datos (Repository)	RF19	Maneja el almacenamiento de comunidades y sus miembros en la base de datos.
SpaceEntity	Persistencia (Entity)	RF19	Representa comunidades o grupos donde los usuarios pueden interactuar.
MessagingController	Presentación (Controller)	RF22	Gestiona las solicitudes de envío y recepción de mensajes privados entre usuarios.

MessagingService	Lógica de negocio (Service)	RF22	Implementa la lógica de comunicación privada entre usuarios.
DirectMessageEntity	Persistencia (Entity)	RF22	Representa los mensajes enviados dentro de conversaciones privadas.
NotificationController	Presentación (Controller)	RF28, RF29	Permite consultar las notificaciones del usuario.
NotificationService	Lógica de negocio (Service)	RF28, RF29	Gestiona la creación y envío de notificaciones relacionadas con retos y actividad social.
NotificationRepository	Acceso a datos (Repository)	RF28, RF29	Maneja el almacenamiento de notificaciones en la base de datos.
NotificationEntity	Persistencia (Entity)	RF28, RF29	Representa las notificaciones generadas por actividades dentro del sistema.

c. Diagrama de paquetes:



(hecho en draw.io)

d. Descripción de cada paquete y sus componentes:

En NestJS, los paquetes son los módulos:

- auth
- users
- exercises
- routines
- workouts
- challenges
- social
- spaces
- messaging

- notifications

Cada uno agrupa:

- controllers
- services
- repositories
- entities
- dto

Auth

Gestiona la autenticación de los usuarios del sistema mediante JWT. Incluye las clases necesarias para el registro, inicio de sesión y validación de credenciales.

Componentes:

- AuthController
- AuthService
- JwtStrategy
- DTOs de autenticación

Users

Gestiona la información de los usuarios del sistema, incluyendo el perfil, preferencias y configuración personal.

Componentes:

- UsersController
- UsersService
- UserRepository
- UserEntity
- UserProfileEntity

Exercises

Contiene el catálogo de ejercicios disponibles en la aplicación y la lógica asociada a su consulta y clasificación.

Componentes:

- ExercisesController
- ExercisesService
- ExerciseEntity
- ExerciseCategoryEntity
- BodyPartEntity
- MetricTypeEntity

Routines

Gestiona las rutinas de entrenamiento reutilizables creadas por los usuarios.

Componentes:

- RoutinesController
- RoutinesService
- RoutineEntity
- RoutineExerciseEntity
- RoutineExerciseSetEntity

Workouts

Se encarga de registrar las sesiones reales de entrenamiento realizadas por los usuarios.

Componentes:

- WorkoutsController
- WorkoutsService
- WorkoutLogEntity
- WorkoutLogExerciseEntity
- WorkoutLogSetEntity

Challenges

Permite crear, gestionar y participar en retos de actividad física.

Componentes:

- ChallengesController
- ChallengesService
- ChallengeEntity
- ChallengeInviteEntity
- ChallengeCycleDayEntity

Social

Gestiona las publicaciones de progreso y las interacciones sociales entre usuarios.

Componentes:

- SocialController
- SocialService
- WorkoutPostEntity
- WorkoutPostLikeEntity

Spaces

Permite crear comunidades o grupos donde los usuarios pueden comunicarse.

Componentes:

- SpacesController
- SpacesService
- SpaceEntity
- SpaceMemberEntity
- SpaceMessageEntity

Messaging

Gestiona las conversaciones privadas entre usuarios.

Componentes:

- MessagingController
- MessagingService
- DirectConversationEntity
- DirectMessageEntity

Notifications

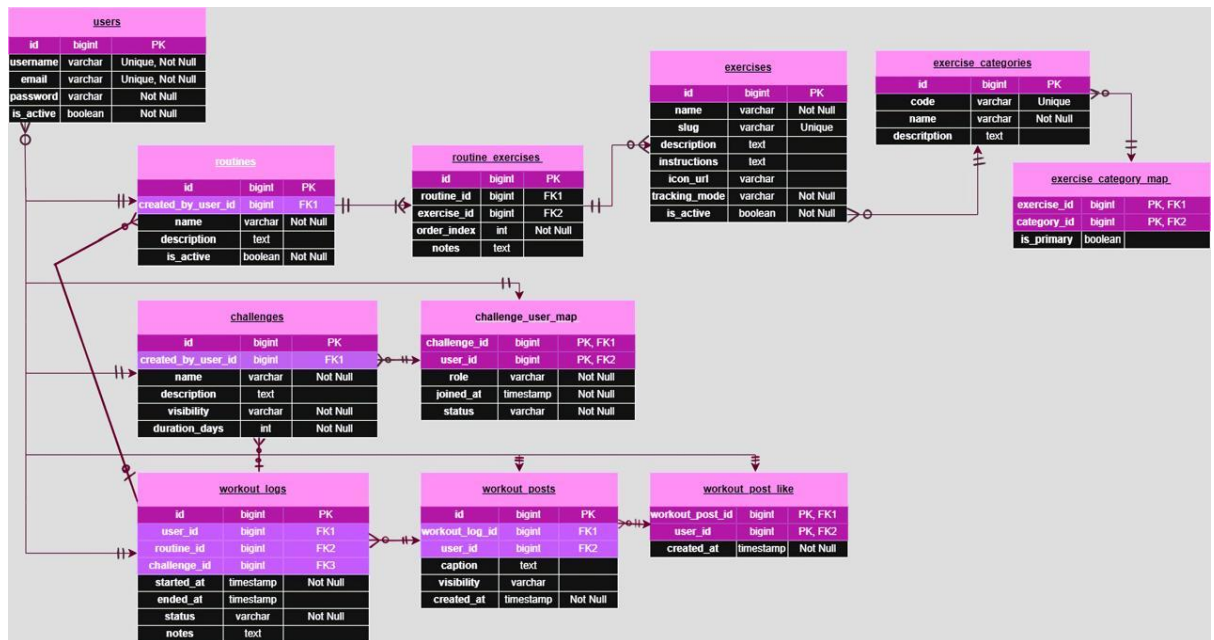
Gestiona las notificaciones generadas por las actividades del sistema.

Componentes:

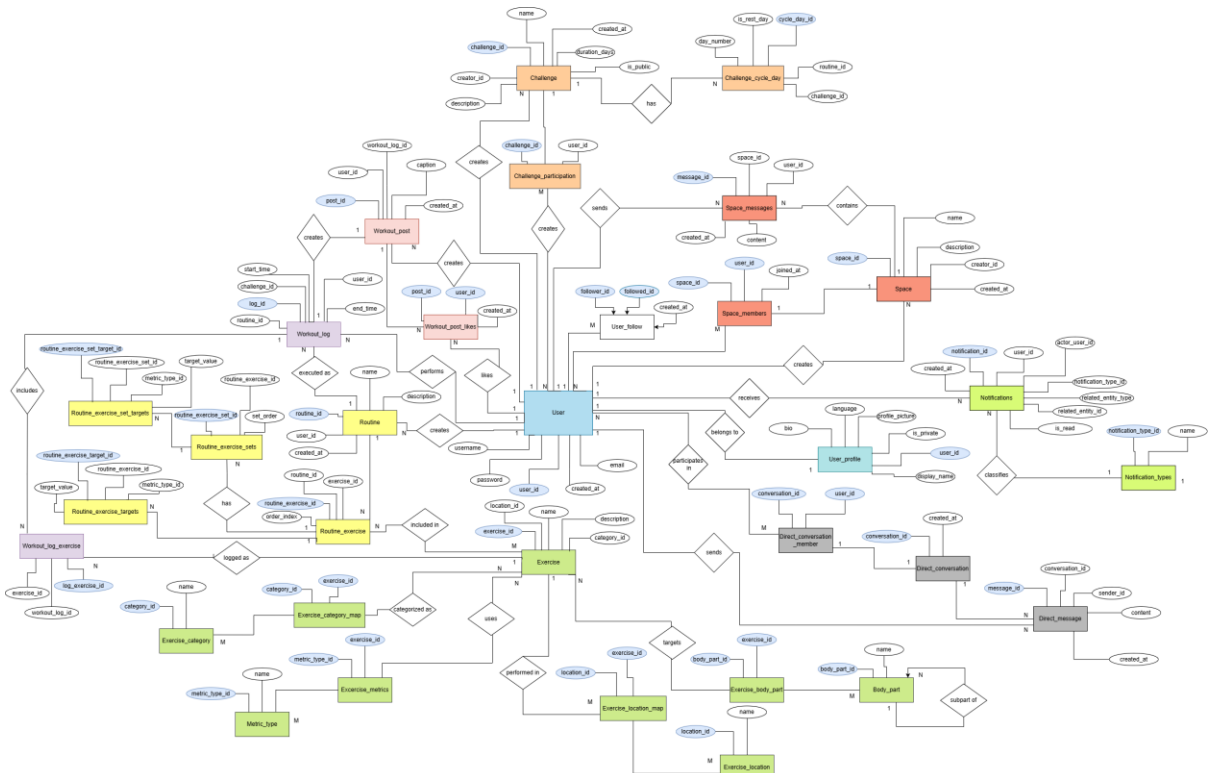
- NotificationsService
- NotificationEntity
- NotificationTypeEntity

4. Persistencia de datos:

a. Diagrama de clases persistentes



b. Diagrama Entidad-Relación



Todas las imágenes de los diagramas se encuentran en la carpeta: Diagramas y prototipos en la carpeta del corte en el repositorio

5. Selección de tecnología:

a. Estimaciones de las dimensiones que podría tomar el sistema:

Se dividió el alcance del sistema en estas categorías acorde a los requisitos no funcionales:

- Gestión de usuarios y perfiles
- Catálogo de ejercicios
- Creación y participación en retos
- Registro de entrenamientos (workouts)
- Interacción social (publicaciones, spaces, mensajería)
- Sistema de notificaciones

No se contemplan funcionalidades avanzadas como integración con dispositivos biométricos o sensores externos.

Gestión de usuarios y perfiles

Se encarga del registro, autenticación y administración de perfiles de usuario dentro del sistema.

Se estima que el sistema podrá manejar:

- Entre 500 y 1,000 usuarios en una fase inicial
- Hasta 10,000 usuarios en crecimiento medio
- Un máximo estimado de 50,000 usuarios en escenarios de alta adopción
- En términos de concurrencia, se espera que entre un 5% y 10% de los usuarios estén activos simultáneamente.
- El tiempo de respuesta esperado para operaciones como registro e inicio de sesión es de 1 a 2 segundos.

Catálogo de ejercicios

Almacena y gestiona los ejercicios disponibles en el sistema, incluyendo sus categorías, métricas y relaciones con partes del cuerpo.

Se estima que el catálogo contendrá:

- Entre 100 y 500 ejercicios iniciales
- Posibilidad de crecimiento a 1,000+ ejercicios
- El acceso a esta información debe mantenerse con tiempos de respuesta de menos de 2 segundos, ya que es consultado frecuentemente.

Creación y participación en retos

Este módulo permite a los usuarios crear, explorar y unirse a retos de actividad física.

Se estima que:

- Cada usuario puede participar en 1 a 5 retos activos simultáneamente
- Un reto puede tener entre 5 y 100 participantes
- El sistema debe soportar múltiples relaciones entre usuarios y retos sin afectar el rendimiento.
- Las operaciones principales (crear reto, unirse, consultar información) deben responder en un máximo de 2 a 3 segundos.

Registro de entrenamientos

Gestiona el registro de la actividad física realizada por los usuarios.

Se considera que:

- Cada usuario puede generar múltiples registros diarios
- Cada workout puede incluir varios ejercicios y serie

Se estima que:

- Un usuario puede generar entre 1 y 5 registros diarios, debido a que un challenge solo permite un registro diario
- A mediano plazo, el sistema podría almacenar millones de registros de entrenamiento entre todos los usuarios

Esto implica un crecimiento considerable del almacenamiento, estimado en:

- 1 a 5 MB por usuario
- Hasta 50 GB o más en escenarios de crecimiento

Interacción social (publicaciones, spaces, mensajería)

Permite la comunicación e interacción entre usuarios mediante publicaciones, espacios comunitarios (spaces) y mensajes privados.

Se estima que:

- Cada usuario puede generar múltiples publicaciones y mensajes
 - Los espacios pueden agrupar entre 10 y 100 usuarios
- Este módulo puede generar un alto volumen de datos, especialmente en: mensajes, publicaciones e interacciones

Por lo tanto, se requiere que las consultas se mantengan en un rango de 2 a 3 segundos.

Sistema de notificaciones

Gestiona el envío de notificaciones relacionadas con la actividad del sistema.

Se considera que:

- Cada usuario puede recibir múltiples notificaciones diarias
- El sistema debe procesar eventos en tiempo casi real

El tiempo de entrega de notificaciones debe ser:

- menor a 2 segundos en condiciones normales

Consideraciones generales del sistema

Escalabilidad

El sistema está diseñado para escalar mediante:

- arquitectura modular
- uso de contenedores Docker
- separación entre backend y base de datos

Esto permite aumentar la capacidad del sistema conforme crece el número de usuarios y datos.

Disponibilidad

Se espera que el sistema tenga una disponibilidad de al menos 99% de disponibilidad

Confiabilidad

Para garantizar la integridad de la información se contemplan:

- respaldos periódicos de la base de datos
- posibilidad de replicación en PostgreSQL
- manejo de errores en el backend

b. Descripción de tecnologías

Frontend: React Native

Se eligió React Native porque permite desarrollar la aplicación móvil de forma más eficiente utilizando una sola base de código, lo cual facilita el desarrollo del proyecto considerando el tiempo y alcance disponible

Ventajas

- Permite desarrollar aplicaciones móviles multiplataforma (Android e iOS) con una sola base de código.
- Utiliza componentes reutilizables, lo que facilita la organización del proyecto.
- Tiene buena integración con APIs REST, lo cual es ideal para conectarse con el backend en NestJS.
- Cuenta con una gran comunidad y documentación.

Desventajas

- Algunas funcionalidades específicas del dispositivo pueden requerir librerías adicionales.
- El rendimiento puede ser menor comparado con aplicaciones nativas en casos muy complejos.

Framework Backend: NestJS

Se eligió NestJS porque proporciona una estructura organizada desde el inicio, lo que facilita mantener el proyecto ordenado a medida que crece en complejidad

Ventajas

- Permite trabajar de forma modular, lo cual facilita separar el sistema por funcionalidades (usuarios, retos, ejercicios, etc.).

- Tiene soporte para inyección de dependencias, lo que ayuda a mantener el código ordenado y desacoplado.
- Facilita la implementación de patrones como Repository, Service y DTO, que ya vienen bastante integrados.
- Se integra bien con bases de datos relacionales mediante ORMs como TypeORM.
- Es compatible con Docker, lo que facilita su despliegue.
- Permite construir APIs REST de forma clara para aplicaciones móviles

Desventajas

- Es un framework relativamente nuevo comparado con otras opciones que habíamos considerado
- Requiere entender su estructura al inicio, lo cual puede tomar un poco de tiempo.

Lenguaje de programación: TypeScript

Se utilizará TypeScript para tener un mayor control sobre los datos y evitar errores durante el desarrollo, especialmente en nuestro sistema con múltiples entidades y relaciones

Ventajas

- El tipado ayuda a evitar errores durante el desarrollo.
- Hace que el código sea más fácil de mantener en proyectos grandes.
- Se integra naturalmente con NestJS.
- Es el lenguaje recomendado para trabajar con NestJS

Desventajas

- Requiere una compilación previa antes de ejecutar

ORM: TypeORM

Se eligió TypeORM porque permite trabajar la base de datos de forma más intuitiva dentro del código, facilitando la conexión entre las entidades y la base de datos

Ventajas

- Se integra directamente con NestJS.

- Permite trabajar con la base de datos usando objetos, lo cual simplifica el desarrollo.
- Facilita el uso del patrón Repository.

Desventajas

- A medida que el sistema crece, la configuración de relaciones complejas y consultas avanzadas puede volverse más detallada y requerir un mayor entendimiento del ORM.

Seguridad: JWT

Se decidió utilizar JWT para manejar la autenticación de usuarios de forma sencilla y compatible con aplicaciones móviles, sin depender de sesiones en el servidor

Ventajas

- Permite autenticar sin necesidad que el servidor tenga que guardar información de sesión de cada usuario.
- Funciona bien con aplicaciones móviles.
- Permite manejar roles y permisos de usuario.

Desventajas

- Manejar correctamente la expiración y renovación de tokens

Despliegue: Docker

Se utilizará Docker para facilitar la ejecución del sistema en distintos entornos sin problemas de configuración.

Ventajas:

- Permite ejecutar el sistema en distintos entornos sin problemas de configuración.
- Facilita levantar tanto el backend como la base de datos con Docker Compose.
- Mantiene consistencia entre desarrollo y producción.

Desventajas:

- Requiere configuración inicial de contenedores.

Arquitectura Modular por Capas

El sistema se organiza utilizando una arquitectura modular por capas, lo que significa que el sistema se divide en módulos (usuarios, ejercicios, rutinas, etc.) y dentro de cada módulo se separa el código por responsabilidades

Las capas utilizadas son:

- Capa de Presentación (Controllers)
Manejan las solicitudes HTTP y la comunicación con el cliente.
- Capa de Negocio (Services)
Contienen la lógica principal del sistema.
- Capa de Acceso a Datos (Repositories)
Se encargan de interactuar con la base de datos.
- Capa de Persistencia (Entities)
Representan las tablas de la base de datos.

Ventajas

- Permite mantener el código organizado.
- Hace más fácil agregar nuevas funcionalidades.
- Reduce el acoplamiento entre partes del sistema.

Desventajas

- Requiere definir bien la estructura desde el inicio.
- Puede generar muchos archivos en el proyecto.

Patrones:

- Repository: Permite separar la lógica de acceso a datos del resto del sistema.
- Service Layer: Centraliza la lógica de negocio.
- DTO (Data Transfer Object): Permite controlar la información que entra y sale del sistema.
- Dependency Injection: Facilita la comunicación entre componentes sin generar dependencias fuertes.
-

c. Tecnología para almacenamiento persistente del gestor de base de datos

Base de Datos: PostgreSQL

Se eligió PostgreSQL porque se adapta bien a un modelo relacional como el del sistema, donde existen múltiples relaciones entre entidades

Ventajas

- Soporta relaciones complejas entre tablas, lo cual es necesario para el modelo del sistema.

- Es altamente estable y ampliamente utilizada en entornos reales.
- Se integra correctamente con TypeORM.
- Permite manejar grandes volúmenes de datos de forma eficiente.

Desventajas

- Puede requerir configuración inicial para su despliegue.
- Algunas consultas complejas pueden requerir optimización.

6. Planificación y gestión:

a. Lista de tareas

Tarea	Responsable
Refinamiento de prototipos finales (v3)	Camila Sandoval
Diseño del Dashboard (visualización de progreso)	Camila Sandoval
Diseño de componentes reutilizables de interfaz	Camila Sandoval
Pruebas de usabilidad de interfaz	Camila Sandoval
Diseño del Buscador de Challenges con filtros	Emily Góngora
Diseño de Perfil de Usuario	Emily Góngora
Diseño de Chat y Espacios comunitarios	Emily Góngora
Diseño de experiencia de invitación de amigos	Emily Góngora
Implementación de interfaz de registro e inicio de sesión	Camila Sandoval
Implementación de interfaz de creación y exploración de retos	Emily Góngora
Diseño del modelo entidad-relación	Esteban de la Peña
Definición de estructura de retos y progreso	Esteban de la Peña
Normalización e integridad de datos	Esteban de la Peña
Documentación técnica de la base de datos	Esteban de la Peña
Definición de relaciones entre usuarios y grupos	Martin Villatoro
Definición de roles del sistema	Martin Villatoro
Diseño de estructura de notificaciones	Martin Villatoro
Elaboración de consultas clave de progreso	Martin Villatoro
Implementación del esquema de base de datos en PostgreSQL	Esteban de la Peña
Configuración de integridad referencial y restricciones	Martin Villatoro
Configuración inicial del backend (NestJS, dependencias y estructura de proyecto)	Alejandra Sierra
Definición de arquitectura del backend (arquitectura en capas y módulos)	Alejandra Sierra
Configuración de conexión a base de datos (TypeORM + PostgreSQL)	Alejandro Pérez
Implementación de autenticación (registro e inicio de sesión)	Alejandra Sierra
Implementación de autorización basada en roles del sistema	Alejandra Sierra
Implementación de creación y unión a retos	Alejandra Sierra
Implementación de registro de métricas y fotografías	Alejandra Sierra
Implementación de control de privacidad	Alejandra Sierra

Implementación de sistema de notificaciones	Alejandra Sierra
Implementación de exploración y filtrado de retos	Alejandro Pérez
Implementación de invitación de amigos	Alejandro Pérez
Implementación de historial de actividades	Alejandro Pérez
Implementación de visualización de progreso colectivo	Alejandro Pérez
Integración entre frontend y backend	Alejandro Pérez
Configuración de contenedores para despliegue (Docker)	Alejandro Pérez
Pruebas funcionales por módulo	Equipo completo
Validación de historias de usuario	Equipo completo
Ajustes posteriores al testeo	Equipo completo
Pruebas de integración del sistema completo	Equipo completo
Elaboración de bitácora de reuniones	Equipo completo
Gestión del tiempo (LOGT individual)	Equipo completo
Seguimiento del avance del proyecto y coordinación del equipo	Equipo completo

b. Reflexión informe LOGT grupal

En general, a pesar de que todos participamos activamente y cumplimos con nuestras responsabilidades en su totalidad, nos atrasamos en la realización de la documentación y diagramas. Tuvimos reuniones en las que discutimos los prototipos, las tecnologías, la estructura del sistema, las entidades, cómo almacenaríamos la información en la base de datos, etc. Pero no lo anotamos en el documento al mismo tiempo, para la próxima entrega lo mejor es designar a un miembro del equipo que vaya redactando las decisiones tomadas en el documento a entregar así no hay que repetir el trabajo de última hora.